

M.C.A. (Management)

JPR 551 MJ: Java Programming (2024 Pattern) (Semester - II)

Time : 2½ Hours]

[Max. Marks : 50]

Instructions to the candidates:

- 1) *All questions are compulsory.*
- 2) *Figures to the right indicate full marks.*

Q1) Attempt any two :

**a) Define class student with suitable attributes and methods.
Display the student details.**

[5]

Answer:

1. Aim

- To create a Java class **Student** with suitable attributes and methods and display student details.

2. Theory

- A **class** in Java is a blueprint used to create objects.
- It contains **data members (variables)** and **member functions (methods)**.
- In this program:
 - Attributes: name, roll number, and marks
 - Methods:
 - getData() → to accept student details
 - displayData() → to display student details

3. Program

```
import java.util.Scanner;

class Student
{
    String name;
    int rollNo;
    float marks;

    void getData()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter Name:");
        name = sc.nextLine();

        System.out.println("Enter Roll No:");
        rollNo = sc.nextInt();

        System.out.println("Enter Marks:");
        marks = sc.nextFloat();
    }

    void displayData()
    {
        System.out.println("Student Details:");
        System.out.println("Name: " + name);
        System.out.println("Roll No: " + rollNo);
        System.out.println("Marks: " + marks);
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Student s = new Student();
        s.getData();
        s.displayData();
    }
}
```

4. Output

Enter Name:

Yogita

Enter Roll No:

101

Enter Marks:

90

Student Details:

Name: Yogita

Roll No: 101

Marks: 90.0

5. Explanation

- A class **Student** is created with three variables: name, roll number, and marks.
- The method `getData()` takes input from the user.
- The method `displayData()` prints the student details.
- In the main method, an object `s` is created and methods are called to perform operations.

b) Create an abstract class shape with method area(). Inherit this class for creating square class. Display area of square. [5]

Answer:

1. Aim

- To create an abstract class **Shape** and implement its method in **Square** class to calculate and display area.

2. Theory

- An **abstract class** is a class that cannot be instantiated and may contain **abstract methods** (methods without body).
- Abstract method must be implemented in derived (child) class.
- Here, Shape is an abstract class with method `area()`.
- Square class inherits Shape and defines the area formula:

Area of Square = side × side

3. Program

```
abstract class Shape
{
    abstract void area(); // abstract method
}

class Square extends Shape
{
    int side = 5;

    void area()
    {
        int result = side * side;
        System.out.println("Area of Square: " + result);
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Square s = new Square();
        s.area();
    }
}
```

4. Output

Area of Square: 25

5. Explanation

- Shape is declared as an abstract class with abstract method area().
- Square class extends Shape and provides implementation of area().
- Object of Square is created in main method.
- The method calculates and displays area using formula **side × side**.

C) Write a function using lambda expression to calculate to cube of a given number. [5]

Answer:

1. Aim

- To write a Java program using **lambda expression** to calculate cube of a number.

2. Theory

- A **lambda expression** in Java is used to implement a **functional interface** (interface with one abstract method) in a short and simple way.
- It reduces code length and improves readability.

Syntax:

(parameters) -> expression

Here, cube is calculated using:

Cube = number × number × number

3. Program

```
interface Cube
{
    int calculate(int x);
}

public class Main
{
    public static void main(String[] args)
    {
        Cube c = (x) -> x * x * x; // lambda expression

        int result = c.calculate(3);

        System.out.println("Cube is: " + result);
    }
}
```

```
}  
}
```

4. Output

Cube is: 27

5. Explanation

- An interface Cube is created with one method calculate().
- Lambda expression (x) -> x * x * x is used to define the method.
- The value 3 is passed, and cube is calculated.
- Output displays the cube of the number.

d) Explain use of final keyword for class, data members, methods. [5]

Answer:

Definition:

- The final **keyword** in Java is used to **restrict modification**.
- It ensures that certain elements cannot be changed once defined.
- It can be applied to:
 - Variables (data members)
 - Methods
 - Classes

1) Final Variable (Data Member)

- A final variable cannot be changed once assigned.
- It behaves like a **constant**.
- **Short Example:**

```
final int x = 10;  
System.out.println(x);  
// x = 20; // Error
```

Explanation:

Value of x remains fixed throughout the program.

2) Final Method

- A final method cannot be overridden in a subclass.
- **Short Example:**

```
class A
{
    final void show()
    {
        System.out.println("Final Method");
    }
}

class B extends A
{
    // void show() { } // Error
}
```

Explanation:

Subclass cannot change the method implementation.

3) Final Class

- A final class cannot be inherited.
- **Short Example:**

```
final class A { }
```



```
// class B extends A { } // Error
```

Explanation:

No other class can extend class A.

Conclusion

- Final variable → constant value
- Final method → no overriding
- Final class → no inheritance

The final keyword helps in **security, preventing changes, and maintaining fixed behavior.**

Q2) Attempt any two:

a) Explain Thread Life Cycle with suitable diagram.

[5]

Answer:

Definition:

- In Java, a thread goes through different stages during its execution.
- These stages are defined in the Java Programming Language under the Thread class.
- A thread life cycle describes the various states a thread enters from its creation to termination.

States of Thread Life Cycle

1. New State

- A thread is in the **New** state when it is created but not yet started.
- It is created using the Thread class or by implementing the Runnable interface.
- The thread will not execute until the start() method is called.\
- Example:

```
Thread t = new Thread();
```

Here, the thread is in the **New** state.

2. Runnable State:

- When start() method is called, the thread enters the **Runnable** state.
- The thread is ready to run and waiting for CPU time.
- It depends on the thread scheduler to move to Running state.
- t.start();

3. Running State

- The thread is in **Running** state when the CPU starts executing the run() method.
- Only one thread runs at a time.
- After execution, it may move to:Blocked, Waiting,Terminated state.

4. Blocked State:

- The thread will be in blocked state **when it is trying to acquire a lock** but currently the lock is acquired by the other thread.
- The thread will move from the blocked state to runnable state when it acquires the lock.

5. Waiting State

- The thread will be in waiting state **when it calls wait()** method or **join()** method.
- It will move to the runnable state when other thread will notify or that thread will be terminated.

6. Timed Waiting State

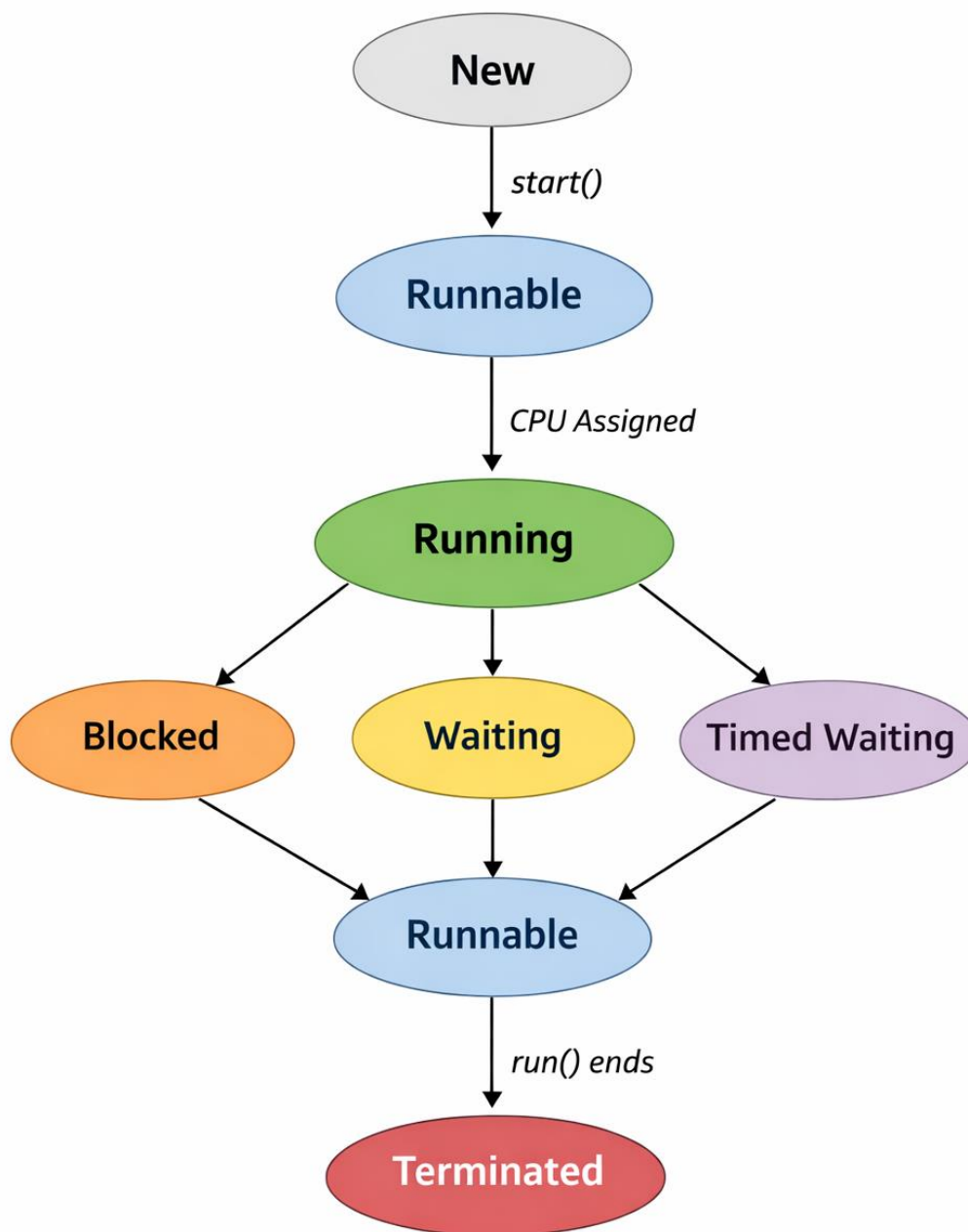
- A thread waits for a specified amount of time.
- Methods: 1. sleep(time), 2. wait(time), 3. join(time)
- Example:

Thread.sleep(2000);

Thread sleeps for 2 seconds.

7. Terminated (Dead) State

- A thread enters **Terminated** state when:
- run() method completes
- Thread finishes execution or stops.
- An exception occurs: A terminated thread cannot be restarted.



b) Accept age from user. If age is negative throw Invalid Age exception. (user defined) [5]

Answer:

1. Aim

- To create a **user-defined exception** in Java and throw it when the entered age is negative.

2. Theory

- An **exception** is an unwanted event that interrupts normal program flow.
- A **user-defined exception** is created by extending the Exception class.
- The throw keyword is used to explicitly throw an exception.
- The try-catch block is used to handle exceptions.

3. Program

```
import java.util.Scanner;

// User-defined exception
class InvalidAgeException extends Exception
{
    InvalidAgeException(String msg)
    {
        super(msg);
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter Age:");
        int age = sc.nextInt();
    }
}
```

```

try
{
    if (age < 0)
    {
        throw new InvalidAgeException("Age cannot be negative");
    }
    System.out.println("Valid Age: " + age);
} catch (InvalidAgeException e)
{
    System.out.println("Exception: " + e.getMessage());
}
}

```

4. Output

Case 1 (Valid Age):

Enter Age:25
Valid Age: 25

Case 2 (Invalid Age):

Enter Age:-5
Exception: Age cannot be negative

5. Explanation

- A class InvalidAgeException is created by extending Exception.
- Age is taken as input from user.
- If age is negative, exception is thrown using throw.
- The exception is handled using catch block and message is displayed.

c) Demonstrate synchronized keyword with example. [5]

Answer:

1. Meaning

- **Synchronization** means **controlling multiple threads so that they do not access shared resources at the same time.**
- When many threads work on the **same data**, problems like **data inconsistency** may occur.
- Synchronization ensures: Only **one thread** accesses the resource at a time.

2. Why Synchronization is Needed

- Example problem: Two threads updating the **same bank account balance.**

Without synchronization:

Thread 1 → withdraw ₹500

Thread 2 → withdraw ₹500

Balance may become **incorrect.**

Synchronization prevents this issue.

3. Using synchronized Keyword

Java provides the synchronized **keyword** to achieve synchronization.

Example:

```
class Test
```

```
{
```

```
    synchronized void show()
```

```
    {
```

```
        System.out.println("Thread is running");
```

```

    }
}

class MyThread extends Thread
{
    Test t;

    MyThread(Test t)
    {
        this.t = t;
    }

    public void run()
    {
        t.show();
    }
}

public class DemoS
{
    public static void main(String args[])
    {
        Test obj = new Test();

        MyThread t1 = new MyThread(obj);
        MyThread t2 = new MyThread(obj);

        t1.start();
    }
}

```

```
        t2.start();
    }
}
```

Output:
Thread is running
Thread is running

Explanation

show() method is **synchronized**.

Only **one thread can execute the method at a time**.

The second thread waits until the first thread finishes.

d) Write a program to demonstrate HashSet. Create a HashSet and perform operations add and remove elements. [5]

Answer:

1. Aim

- To create a **HashSet** in Java and perform **add** and **remove** operations.

2. Theory

- **HashSet** is a class in Java Collection Framework.
- It is used to store a collection of **unique elements** (no duplicates allowed).
- It does not maintain insertion order.
- It provides methods like add() and remove() to manipulate elements.

Key Points:

- No duplicate values
- Unordered collection
- Uses hashing internally

3. Program

```
import java.util.HashSet;

public class Main
{
    public static void main(String[] args)
    {

        HashSet<String> set = new HashSet<>();

        // Adding elements
        set.add("Apple");
        set.add("Banana");
        set.add("Mango");
        set.add("Apple"); // duplicate (ignored)

        System.out.println("HashSet after adding elements: " +
set);

        // Removing element
        set.remove("Banana");

        System.out.println("HashSet after removing element: " +
set);
    }
}
```


4. Output

HashSet after adding elements: [Apple, Mango, Banana]

HashSet after removing element: [Apple, Mango]

(Order may vary because HashSet is unordered.)

5. Explanation

- A HashSet named set is created.
- Elements are added using add() method. Duplicate element is ignored.
- Element "Banana" is removed using remove() method.
- Final set is displayed.

Q3) Attempt any one

a) Create a Servlet which checks username and password, if it is correct, display message “Successful login,” else display appropriate error message (without JDBC) [10]

Answer:

Objective

Create a servlet that:

- Checks username & password
- If correct → **“Successfully login”**
- If wrong → **Error message**

Step 1: Create Dynamic Web Project in Eclipse

1. Open Eclipse
2. Click: **File** → **New** → **Dynamic Web Project**
3. Enter Project Name: **LoginApp**
4. Select Target Runtime: **Apache Tomcat**
5. Click **Finish**

Step 2: Create HTML Login Page

Right click on project → New → HTML File → name: login.html

```
<!DOCTYPE html>
<html>
<head>
<title>Login Page</title>
</head>
<body>
<h2>Login Form</h2>

<form action="/login" method="post">
Username: <input type="text" name="uname"><br><br>
Password: <input type="password" name="pass"><br><br>
<input type="submit" value="Login">
</form>
</body>
</html>
```

Step 3: Create Servlet Class

Right click → New → Servlet → name: LoginServlet

Servlet Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/login")
public class LoginServlet extends HttpServlet
{

    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException
    {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
```

```
// Get form data
String username = request.getParameter("uname");
String password = request.getParameter("pass");

// Hardcoded values (no database)
if(username.equals("admin") && password.equals("1234"))
{
    out.println("<h2 style='color:green'>Successfully Login</h2>");
}
else
{
    out.println("<h2 style='color:red'>Invalid Username or Password</h2>");
}
}
```

Step 4: Configure Server (Tomcat)

1. Go to **Servers tab**
2. Right click → New → Server
3. Select **Apache Tomcat v9**
4. Add your project → Finish

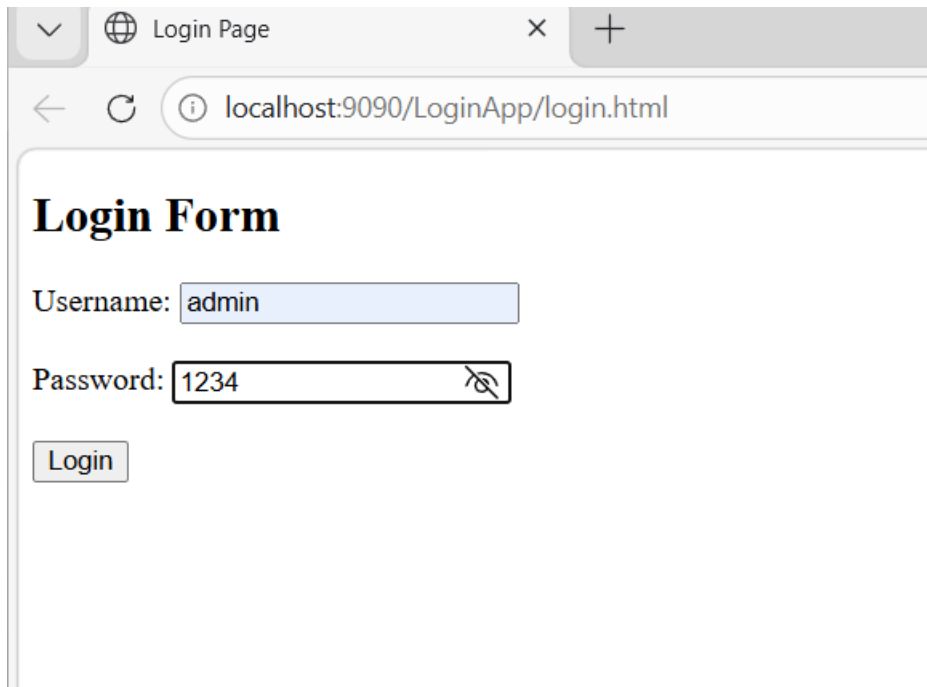
Step 5: Run the Project

1. Right click project → Run on Server
2. Browser opens
3. Open:

<http://localhost:8080/LoginApp/login.html>

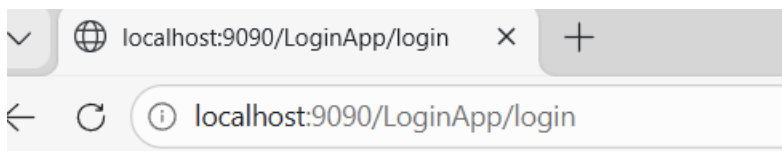
Step 6: Test Output

✓ **Correct Input:**



A screenshot of a web browser window. The title bar shows a single tab labeled "Login Page". The address bar displays "localhost:9090/LoginApp/login.html". The main content area features a "Login Form" with the title in bold. Below the title, there are two input fields: "Username:" with the value "admin" and "Password:" with the value "1234". The password field has a toggle icon (an eye with a slash) to its right. Below these fields is a "Login" button.

Output:



Successfully Login

Wrong Input:

Output: Invalid Username or Password

Program Explanation

- request.getParameter() → gets form data
- doPost() → handles POST request
- PrintWriter → prints output
- No JDBC → values are hardcoded

b) Demonstrate session management using cookies. [10]

Answer:

Definition: Session Management

- Session management using cookies is a method to maintain user data between multiple requests by storing small pieces of information (called cookies) in the user's browser.
- HTTP is a **stateless protocol**, so the server cannot remember the user. Cookies help to maintain the session.

What is Session?

- A session is a continuous interaction between user and server for a limited time.
- Example: Login to a website and browsing pages without logging again.

What is Cookie?

A cookie is a small text file stored in the browser which contains user information like:

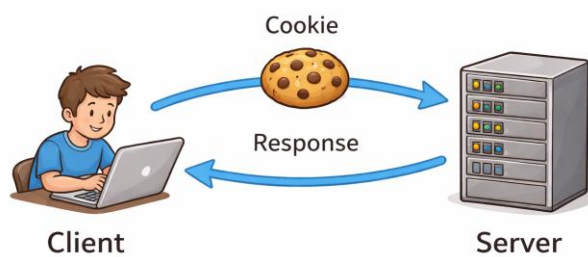
- Username
- User ID
- Login status

Working of Session using Cookies:

1. User sends request to server

2. Server validates user (login)
3. Server creates cookie (username/userID)
4. Cookie is stored in browser
5. Browser sends cookie with every request
6. Server identifies user using cookie

Diagram:



Next Request:

Client → Request + Cookie → Server



Example (Servlet Code):

Creating Cookie:

```
Cookie c = new Cookie("user","Yogita");  
response.addCookie(c);
```

Reading Cookie:

```
Cookie[] cookies = request.getCookies();
for(Cookie c : cookies)
{
    if(c.getName().equals("user"))
    {
        out.println("Welcome " + c.getValue());
    }
}
```

Advantages:

- ✓ Easy to implement
- ✓ Maintains session
- ✓ Improves user experience

Disadvantages:

- ✗ Limited size (4KB)
- ✗ Less secure
- ✗ Can be disabled

Conclusion:

Cookies help in session management by storing user information in the browser, allowing the server to recognize users and provide continuous service.

Q4) Attempt any one :

a) Create a jsp page to accept roll no. of a student, update mobile number in the student table. Display the updated details. Assume suitable structure. [10]

Answer:

1. Aim

To create a JSP application that accepts **roll number**, updates **mobile number** in database, and displays updated student details.

2. Requirements

- Java JDK
- Eclipse IDE
- Apache Tomcat
- MySQL Workbench
- MySQL Connector/J

3. Database Design

Step 1: Use Database

USE test;

Step 2: Create Table (Student)

```
CREATE TABLE student (  
    rollno INT PRIMARY KEY,  
    name VARCHAR(100),  
    mobile VARCHAR(15),  
    course VARCHAR(50)  
);
```


Step 3: Insert Sample Data

```
INSERT INTO student VALUES  
(1,'Sachin','9876543210','MCA'),  
(2,'Ravi','9123456780','BCA');
```

4. Project Creation in Eclipse

Steps:

1. Open Eclipse
2. File → New → Dynamic Web Project
3. Project Name: StudentApp
4. Select Apache Tomcat
5. Finish

5. Add JDBC Driver

- Copy mysql-connector-j-9.6.0.jar
- Paste into:

WEB-INF/lib

Step-by-Step Procedure:

Method 1: Add JAR to Build Path

1. Right click your project
2. Click:
Build Path → Configure Build Path
3. Go to:
Libraries tab - select ClassPath
4. Click:
Add External JARs
5. Select your file:

mysql-connector-j-9.6.0.jar

Click:
Apply → OK

✓ Now check:

Referenced Libraries

→ You should see your JAR file:mysql-connector-j-9.6.0.jar

5. JSP Page (Input Form)

update.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<body>

<h2>Update Mobile Number</h2>

<form action="updateProcess.jsp" method="post">
Roll No: <input type="text" name="rollno"><br><br>
New Mobile: <input type="text" name="mobile"><br><br>

<input type="submit" value="Update">

</form>

</body>
</html>
```

6. JSP Processing Page (Update + Display)

updateProcess.jsp

```
<%@ page import="java.sql.*" %>

<%
int rollno = Integer.parseInt(request.getParameter("rollno"));
String mobile = request.getParameter("mobile");

try
{
Class.forName("com.mysql.cj.jdbc.Driver");
```

```

Connection con =
DriverManager.getConnection( "jdbc:mysql://localhost:3306/test", "root",
"root" );

// □ Update Query
PreparedStatement ps = con.prepareStatement("UPDATE student SET mobile=?
WHERE rollno=?" );

ps.setString(1, mobile);
ps.setInt(2, rollno);

int i = ps.executeUpdate();

if(i > 0)
{
out.println("<h3>Mobile Updated Successfully</h3>");

// □ Fetch Updated Record
PreparedStatement ps2 = con.prepareStatement( "SELECT * FROM student
WHERE rollno=?");

ps2.setInt(1, rollno);
ResultSet rs = ps2.executeQuery();

while(rs.next())
{
%>

<h3>Updated Details:</h3>
Roll No: <%= rs.getInt("rollno") %><br>
Name: <%= rs.getString("name") %><br>
Mobile: <%= rs.getString("mobile") %><br>
Course: <%= rs.getString("course") %><br>

<%
}
}
else
{
out.println("<h3>Record Not Found</h3>");
}

con.close();

```

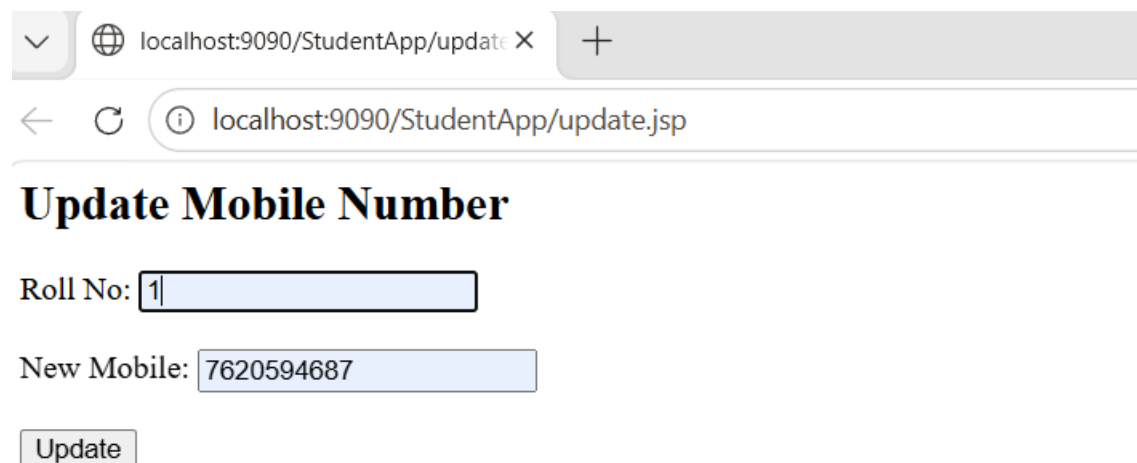
```
} catch(Exception e){  
out.println(e);  
}  
%>
```

7. Execution Steps

1. Right click project → Run on Server
2. Select Apache Tomcat
3. Open:

<http://localhost:8080/YourProjectName/update.jsp>

8. Output

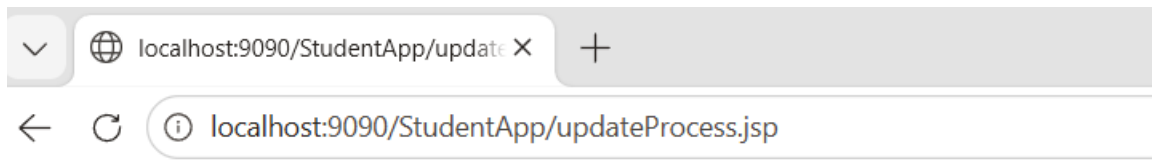


The screenshot shows a web browser window with the address bar displaying 'localhost:9090/StudentApp/update.jsp'. The page title is 'Update Mobile Number'. Below the title, there are two input fields: 'Roll No:' with the value '1' and 'New Mobile:' with the value '7620594687'. At the bottom, there is an 'Update' button.

Update Mobile Number

Roll No:

New Mobile:



Mobile Updated Successfully

Updated Details:

Roll No: 1

Name: Sachin

Mobile: 7620594687

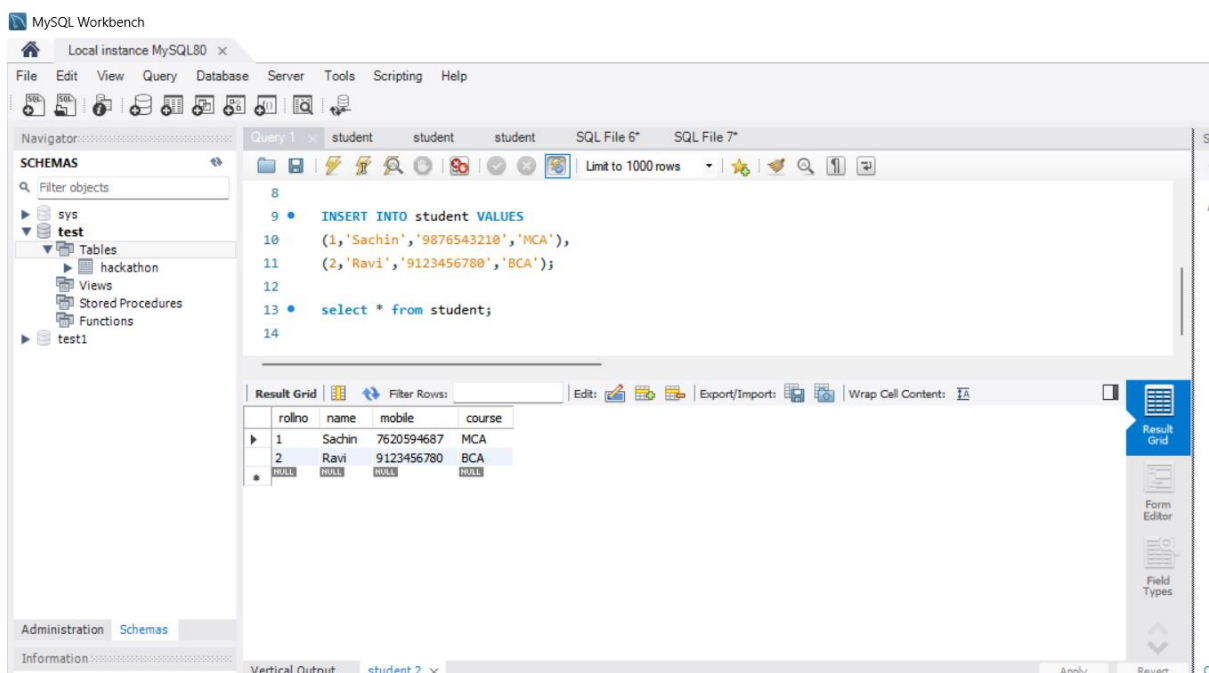
Course: MCA

9. Verification in Database

Open MySQL Workbench

Run:

```
SELECT * FROM student;
```



b) Explain any 5 implicit objects of JSP.

[10]

Answer:

Introduction

- Implicit objects in JSP are **built-in objects** that are automatically created by the JSP container (like Apache Tomcat).
- These objects help in handling **client requests, server responses, session tracking, and output generation** without explicitly creating them in code.
- They are available in all JSP pages by default.

1. request Object

Definition

- The request **object** is used to **receive data from the client** (browser).
- Whenever a user submits a form or sends data via URL, it is stored in the request object.
- It is an instance of:
 HttpServletRequest

What it does

- Gets form data
- Reads user input
- Retrieves request parameters
- Shares data between JSP pages

Common Methods

Method	Description
getParameter()	Gets form data
getParameterValues()	Gets multiple values
getAttribute()	Gets server-side data
setAttribute()	Stores data

Example (Getting Form Data)

```
<%  
String name = request.getParameter("username");  
out.println("Welcome " + name);  
%>
```

If user enters:

username = Yogita

Output:

Welcome Yogita

Real-life Use

- Login forms
- Registration forms
- Search inputs

2. response Object

Definition

- The response **object** is used to **send data back to the client** (browser).
- It controls how the server responds to the browser.
- It is an instance of:
HttpServletResponse

What it does

- Sends output to browser
- Redirects user to another page
- Sets content type

Common Methods

Method	Description
sendRedirect()	Redirect to another page
setContentType()	Sets response type
addCookie()	Adds cookies

Example (Redirect)

```
<%  
response.sendRedirect("welcome.jsp");  
%>
```

This will redirect user to **welcome.jsp**

Example (Content Type)

```
<%  
response.setContentType("text/html");  
out.println("Hello User");  
%>
```

Real-life Use

- Redirect after login
- Sending file download
- Setting response format (HTML, JSON)

3. out Object

Type: JspWriter

Scope: Page

Explanation:

- The out object is used to **display data directly on the browser**.
- It works like System.out.println() but prints on web page instead of console.

Common Methods:

- `println()`
- `print()`
- `flush()`

Basic Syntax

```
out.print("Text");  
out.println("Text");
```

Example:

```
<%  
out.println("Hello JSP");  
out.println("<h2>Welcome User</h2>");  
%>
```

Output

Hello JSP
Welcome User

4. session Object

Definition

- The session object is used to **store user-specific data** across multiple pages.
- Each user has a separate session.
- The **session object** represents a **single user session**.
- It stores data specific to one user.
- It is an instance of:
 `HttpSession`

Key Features

- Unique for each user
- Created when user visits the site
- Destroyed after logout or timeout
- Used for **user-specific data**

Common Methods:

- `setAttribute()` → store data
- `getAttribute()` → retrieve data
- `invalidate()` → destroy session

Syntax

```
session.setAttribute("key", value);  
session.getAttribute("key");
```

Example

```
<%  
session.setAttribute("username", "Yogita");  
%>  
  
<p>  
Welcome <%= session.getAttribute("username") %>  
</p>
```

Output

Welcome Yogita

Real-life Use

- Login user information
- Shopping cart data
- User preferences

5. application Object

Definition

- The **application object** represents the entire web application.
- It is shared by **all users and all sessions**.
- It is an instance of:
 `ServletContext`

Key Features

- Available to **all users**
- Created when the application starts
- Destroyed when the application stops
- Used for **global data sharing**

Common Methods:

- `setAttribute()`
- `getAttribute()`
- `removeAttribute()`

Syntax

```
application.setAttribute("key", value);  
application.getAttribute("key");
```

Example

```
<%  
application.setAttribute("msg", "Welcome to JSP Application");  
%>  
  
<p>  
<%= application.getAttribute("msg") %>  
</p>
```

Output

Welcome to JSP Application

Real-life Use

- Website visitor counter
- Global configuration data
- Shared resources (DB connection info)

Q5) Attempt any one

a) Create a spring mvc form to read registration details for placement drive with spring validation and display it. [10]

Answer:

STEP 1: Create Spring Boot Project

1. Open IntelliJ
2. Click **New Project**
3. Select **Spring Boot**

Fill details:

- Name: placementdrive
- Language: Java
- Packaging: Jar

Add Dependencies:

- ✓ Spring Web
- ✓ Thymeleaf
- ✓ Validation

Click Create/ finish

STEP 2: Project Structure (Auto Created)

You will see:

- src/main/java
- src/main/resources

STEP 3: Create Model Class

1. Create Package

- Right click on com.placement.placementdrive
- New → Package
- Name: model

2. Create Java Class

- Right click on model
- New → Java Class
- Name: Candidate

Code: Candidate.java

```
package com.placement.placementdrive.model;

import jakarta.validation.constraints.*;

public class Candidate
{

    @NotEmpty(message="Name is required")
    private String name;

    @Email(message="Invalid email")
    private String email;

    @Min(value=18, message="Minimum age is 18")
    private int age;

    @NotEmpty(message="Qualification required")
    private String qualification;

    // Getters and Setters
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public int getAge() { return age; }
    public void setAge(int age) { this.age = age; }

    public String getQualification() { return qualification; }
    public void setQualification(String qualification) { this.qualification =
qualification; }
}
```

STEP 4: Create Controller

1. Create Package

- Right click on com.placement.placementdrive
- New → Package
- Name: controller

2. Create Java Class

- Name: CandidateController

Code: CandidateController.java

```
package com.placement.placementdrive.controller;

import com.placement.placementdrive.model.Candidate;
import jakarta.validation.Valid;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.validation.BindingResult;
import org.springframework.web.bind.annotation.*;

@Controller
public class CandidateController
{

    @GetMapping("/")
    public String showForm(Model model)
    {
        model.addAttribute("candidate", new Candidate());
        return "form";
    }

    @PostMapping("/submit")
    public String submitForm(@Valid @ModelAttribute("candidate") Candidate
candidate,
                            BindingResult result)
    {
```

```

        if(result.hasErrors())
        {
            return "form";
        }

        return "result";
    }
}

```

STEP 5: Create HTML Pages

Path: src/main/resources/templates

Code: form.html

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Placement Drive Registration</h2>

<form th:action="@{/submit}" th:object="${candidate}" method="post">

    Name: <input type="text" th:field="*{name}"/>
    <span th:errors="*{name}"></span><br><br>

    Email: <input type="text" th:field="*{email}"/>
    <span th:errors="*{email}"></span><br><br>

    Age: <input type="text" th:field="*{age}"/>
    <span th:errors="*{age}"></span><br><br>

    Qualification: <input type="text" th:field="*{qualification}"/>
    <span th:errors="*{qualification}"></span><br><br>

    <button type="submit">Submit</button>

</form>

</body>
</html>

```

Code: result.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>

<h2>Submitted Details</h2>

Name: <span th:text="${candidate.name}"></span><br>
Email: <span th:text="${candidate.email}"></span><br>
Age: <span th:text="${candidate.age}"></span><br>
Qualification: <span th:text="${candidate.qualification}"></span><br>

</body>
</html>
```

STEP 6: Run Project

- Run PlacementdriveApplication.java
- Click ► Run

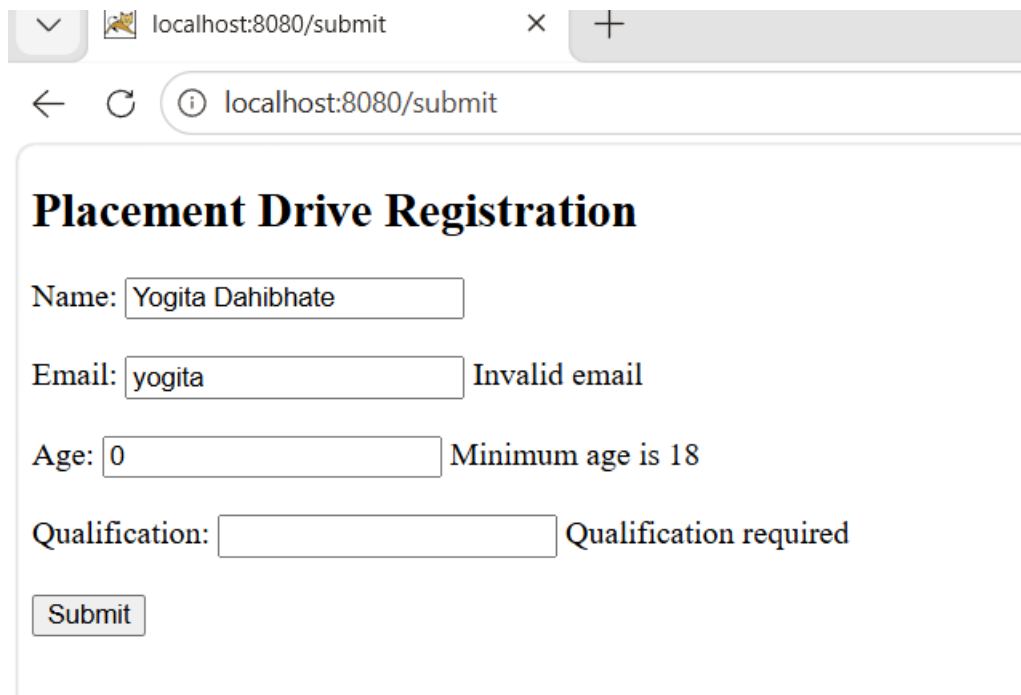
STEP 7: Open in Browser

<http://localhost:8080/>

OUTPUT:

- Form will open
- Enter details
- Validation will check input
- Data displayed on next page

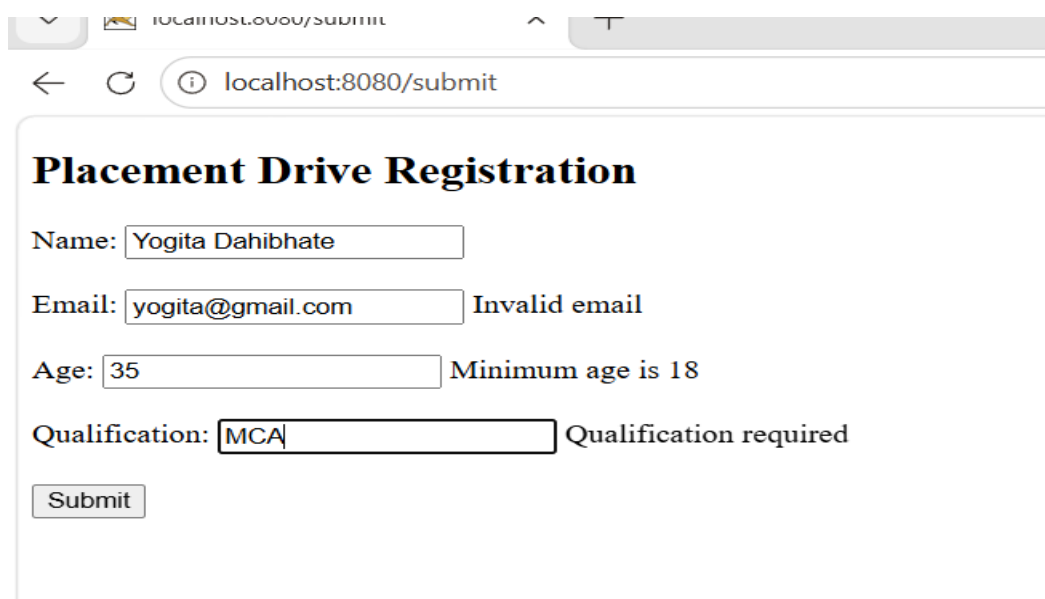
Validation Form:



A screenshot of a web browser window with the address bar showing 'localhost:8080/submit'. The page title is 'Placement Drive Registration'. The form contains the following fields and messages:

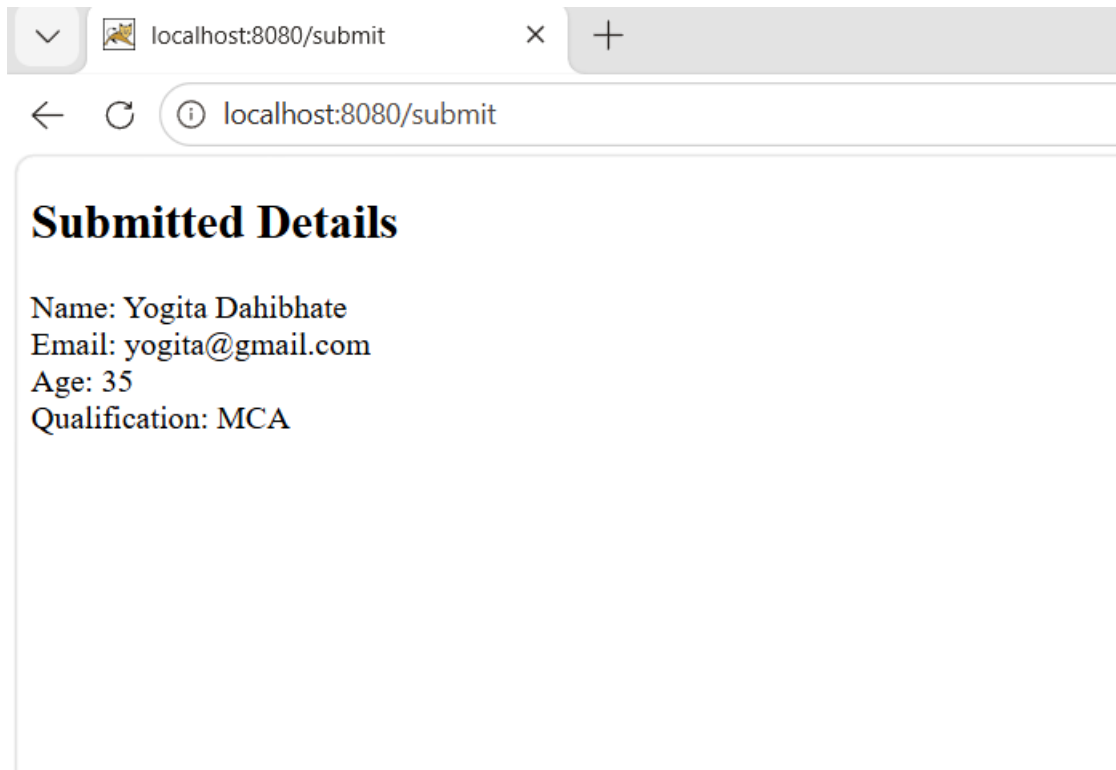
- Name:
- Email: Invalid email
- Age: Minimum age is 18
- Qualification: Qualification required
-

Correct Output:



A screenshot of a web browser window with the address bar showing 'localhost:8080/submit'. The page title is 'Placement Drive Registration'. The form contains the following fields and messages:

- Name:
- Email: Invalid email
- Age: Minimum age is 18
- Qualification: Qualification required
-



b) Explain Spring mvc flow and Inversion of control (IoC) [10]

Answer:

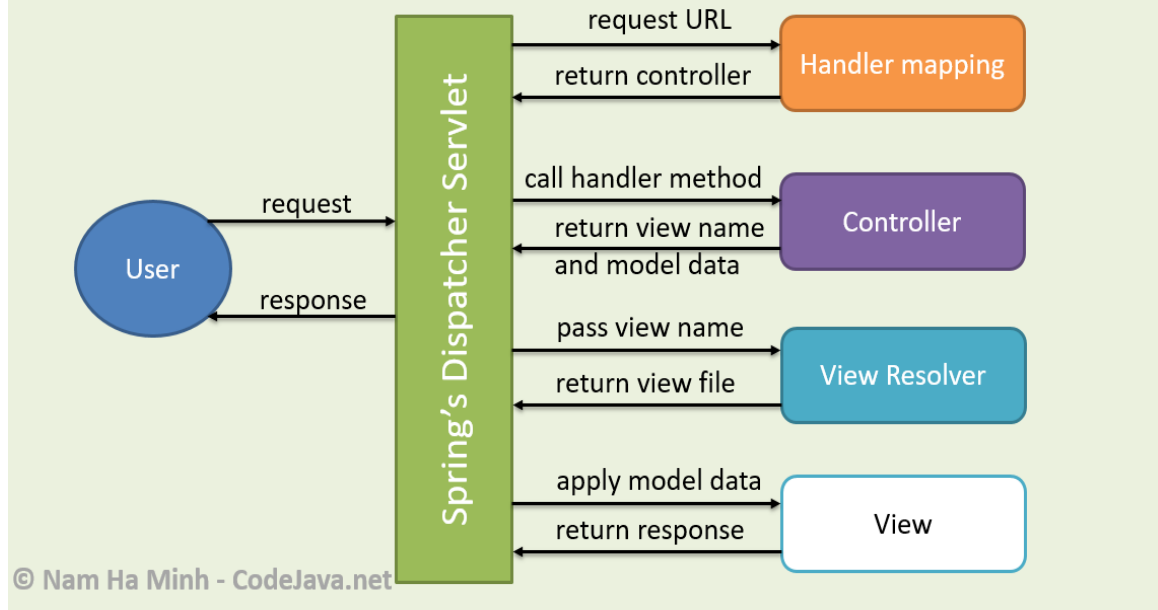
1. Spring MVC Flow:

Introduction

- Spring MVC is a **Java web framework** used to build web applications.
- It follows the **MVC Design Pattern**:
 - **Model** → Data
 - **View** → UI (JSP/HTML)
 - **Controller** → Logic
- It separates application into different parts, making it easy to manage and maintain.

Diagram:

Spring MVC's Architecture



Components of Spring MVC

1. DispatcherServlet

- Central controller (Front Controller)
- Handles all incoming requests
- Controls entire workflow

2. HandlerMapping

- Maps request URL to appropriate controller
- Example: /login → LoginController

3. Controller

- Contains business logic
- Processes user request
- Returns model and view name

4. Model

- Holds data (user info, records, etc.)
- Sent to the view

5. ViewResolver

- Converts logical view name into actual file
- Example: "home" → home.jsp

6. View

- Displays output to user
- Example: JSP, HTML, Thymeleaf page

Step-by-Step Working Flow

1. Client Sends Request

- User enters URL in browser
- Example: http://localhost:8080/home

2. DispatcherServlet Receives Request

- First point of contact
- Acts like a manager

3. DispatcherServlet calls HandlerMapping

- Finds correct controller for request

4. HandlerMapping returns Controller

- Example: HomeController

5. DispatcherServlet calls Controller

- Controller processes request

6. Controller Executes Logic

- Gets data from database/service
- Adds data into Model

7. Controller returns Model + View Name

- Example: ("home", modelData)

8. DispatcherServlet receives response

- Sends it to ViewResolver

9. ViewResolver finds actual View

- Example: home.jsp

10. View Displays Data

- Data shown to user
- Final output appears in browser

2. Inversion of Control (IoC) :

What is IoC?

Definition:

- Inversion of Control (IoC) means the Spring framework creates and manages objects instead of the programmer.
- This makes the code simpler, loosely coupled, and easier to maintain.

Simple Meaning

Normally:

- Programmer creates objects

In IoC:

- Spring creates and manages objects

That's why it is called “**Inversion (reverse) of Control**”

Traditional Way of create object (Without IoC)

```
Car c = new Car();
```

✗Problems:

- Tight coupling
- Hard to maintain
- Difficult to test

Here programmer controls everything

Spring Way of create object (With IoC)

```
// Spring creates object automatically  
Car c; // no 'new' keyword
```

- Here Spring creates object
- Programmer does not create object

What Control is Inverted?

In IoC, the following controls are transferred to Spring:

- Object creation
- Object lifecycle
- Dependency management
- Configuration

IoC Container

- IoC container is the **main component of Spring**
- **It is responsible for:**
 - Creating objects (called Beans)
 - Injecting dependencies
 - Managing lifecycle
 - Configuring objects

Types of IoC Container

1. BeanFactory

- BeanFactory is the **basic IoC container** in Spring used to **create and manage objects (beans)**.
- **Key Points:**
 - **Basic container**
 - Simple and lightweight
 - Provides only basic features of Spring
 - **Uses Lazy Loading**
 - Objects are created **only when needed**
 - Saves memory
 - **Less Features**
 - Mostly used in small applications

2. ApplicationContext

- ApplicationContext is an **advanced IoC container** that provides **all features of BeanFactory + extra features**.
- **Key Points**
 - Advanced container
 - More features (recommended)
 - Used in most applications